Cost Sensitive Learning

we look at cost of misclassification

Predicted \ Actual	N	P
N	C(0,0) TN	C(0,1) FN
P	C(1,0) FP	C(1,1) TP

$$\text{Cost} = C(0,1)\cdot[n_{FN}] + C(1,0)\, n_{FP}$$ } Minimise

if no domain knowledge, invert the imbalance to get cost.

Eg: positive : negative = 1 : 100

⇒ FP cost = C(1,0) = 1

⇒ FN cost = C(0,1) = 100

Cost sensitive resampling: adjust sampling / weights to fit the cost matrix which is skewed.

Extending Decision Trees

CART etc → split based on class label → tries to maximise purity in split

↓

DOES NOT WORK when dataset imbalanced

purity requires P(sample misclassified / split)

↳ weight each class suitably

'weight = "balanced"' → DTC, Scikit-learn

optimal weight: try on training or HPO

ALTERNATE: deal with imbalance first then do this

Extending SVM

SVM hyperplane → heavily biased on majority

Weighted Soft Margin SVM

$C_i = w_i \times C$ → global / class specific

margin less flexible on majority, more on minority

Optimal weight: HPO → GridSearch

Modifying error func: Keras NN

Backprop of error

↳ treats both labels the same

Instead use misclassification error and minimise that.

minority → larger weight

ulta pulta.

'model.fit(Xtrain, Ytrain, class_weight = weights, ...)'

Modify error func: Logistic Regression

$$\text{log loss} = -\frac{1}{N}\sum_{i=1}^{N}\left(y_i \log p_i + (1-y_i)\log(1-p_i)\right)$$

$$\text{weighted log loss} = -\frac{1}{N}\sum_{i=1}^{N}\left(w_0 y_i \log p_i + w_1 (1-y_i)\log(1-p_i)\right)$$

Cost sensitive meta learning: XGBoost

scale_pos_weight: scales gradient of positive class.

for 1:100 imbalance, set to 100 (same inversion)

One Class Classification

no info about minority at all

train only on the majority class

Objective: detect outlier = minority class

Local Outlier Factor (LOF)

1. Define neighbourhood → set k value
2. k-distance(A) = distance to k^{th} (furthest) neighbour
3. reachability distance

$$\text{reach_dist}_k(A,B) = \max\left(k\text{_dist}(B), d(A,B)\right)$$

do for all neighbours and find avg.

stabilise density estimate: "even if A & B are really close, treat their distance as at least B's neighbourhood radius"

4. Local Reachability Density (LRD)

$$LRD(A) = \frac{1}{\text{avg. reach_dist}_k(A)}$$

5. Calculate Local Outlier Factor (LOF)

$$LOF(A) = \frac{\text{avg (LRDs of neighbours)}}{LRD(A)}$$

LOF ≈ 1 → inlier

outlier LOF → outlier!

Isolation Forest

minorities don't have enough friends to go deep into forest.

isolation path = no. of edges from root to leaf

outlier score = avg (isolation path in all iTrees)

One Class SVM

Schölkopf et al.

project to high D → separate data and origin w/ hyperplane

↓

points on origin side = outliers

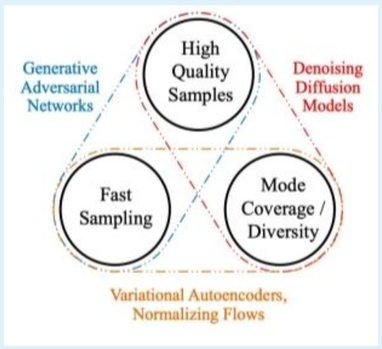
Tax et al.

project → fit hypersphere → minimise volume

↓

points outside = outliers

Synthetic Data vs Augmentation



generative trilemma

- fidelity
- diversity
- sampling speed

} any two, not three

Augment: flip / rotate / scale / add noise

bg noise / pitch / speed / reverb

synonym / random insert, delete / text wraps